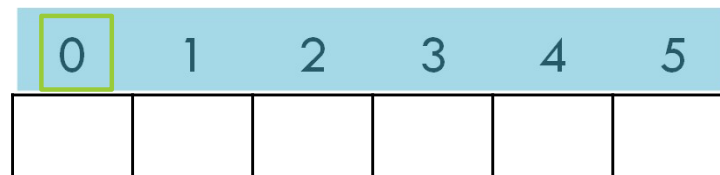


CURSO TÉCNICO SUPERIOR PROFISSIONAL DE PROGRAMAÇÃO DE SISTEMAS DE INFORMAÇÃO
DESENVOLVIMENTO DE APLICAÇÕES

COLEÇÕES E TIPOS GENÉRICOS

ARRAYS (1/5)

- Lista fixa de elementos que podem ser acedidos via index
- É obrigatório especificar o n.º de elementos na inicialização
- O índice do array começa em 0
- Os elementos do array são guardados em endereços de memória seguidos
- Os elementos têm de ser todos do mesmo tipo (ou derivados - herança)
- Tipo por referência o que implica o uso do operador new
 - Precisa de criar o objeto em memória, utilizando depois o endereço de memória para aceder ao mesmo
- Fortemente tipada
- `<tipo>[] <nome> = new <tipo>[tamanho];`



ARRAYS (2/5)

- Declaração e Inicialização

```
string [] colors = new string[2];  
colors[0] = "red";  
colors[1] = "green";
```

```
var cities = new string[]{"Lisboa", "Paris", "Banguécoque"};
```

```
var animals = new Animal[] {new Hiena(), new Animal()};
```

ARRAYS (3/5)

Métodos

A classe Array disponibiliza um conjunto de métodos estáticos para facilitar a realização de determinadas operações e.g.:

- `void Array.Clear(Array array, int index, int length)`
- `void Array.Reverse(Array array)`
- `T Array.Find(Array array, T match)`
- `void Array.Exists(Array array, T match)`
- `void Array.Resize(Array array, int newSize)`
- `int Array.IndexOf(Array array, T value)`
- ...
- *O que faz cada um destes métodos?*

ARRAYS (4/5)

- O array é utilizado quando se sabe previamente o seu tamanho
- Os arrays usam a memória de forma eficiente mas não são flexíveis (relativamente a alterações de tamanho)
- Deve ser declarado com nomes no plural e.g. `veiculos`, `listaDocumentos`
- `foreach` é utilizado para iterar sobre todos os elementos só para leitura
- `for` é utilizado para fazer alterações num subconjunto de elementos
- Podem ser multidimensionais, usando a seguinte sintaxe:

```
<tipo>[,] <nome> = new <tipo>[numLinhas, numColunas];
```

(exemplo bidimensional)

ARRAYS (5/5)

- Utilização de **foreach** – Apenas leitura

```
foreach (string city in cities)
{
    Console.WriteLine(city);
}
```

- Utilização de **for** – Leitura e escrita

```
for (int i = 0; i < cities.Length; i++)
{
    Console.WriteLine(cities[i]);
    cities[i] = cities[i] + " City";
}
```

COLEÇÕES E TIPOS GENÉRICOS

- Fornecem uma forma mais flexível de trabalhar com grupos de objetos
- A memória é gerida dinamicamente
- É possível, em algumas coleções, definir chaves para os objetos, de forma a otimizar as pesquisas
- As **coleções são classes** - por isso, é necessário instanciá-las
- Se a coleção de objetos for apenas de um tipo, devem ser utilizadas as coleções genéricas, que impõem a integridade e segurança do tipo de objeto
- Estão disponíveis nos seguintes namespaces: **System.Collections** e **System.Collections.Generic**

TIPOS GENÉRICOS (1/5)

- Permitem a reutilização do mesmo código por diferentes tipos
- Por exemplo, a classe `List<T>` suporta o tipo `Person`

```
var people = new List<Person>()
{
    new Person(){Name = "Jane", Age = 57},
    new Person(){Name = "Tarzan", Age = 61}
};

foreach (var person in people)
    Console.WriteLine(person.ToString());

public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
    public override string ToString() => $"Name: {Name} Age:{Age}";
}
```

mas também o tipo `int`, o tipo `string` e todos os demais tipos

```
var numbers = new List<int>();

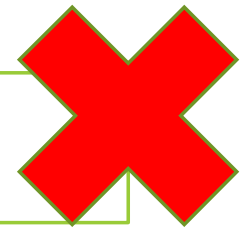
var countries = new List<string>();
```


TIPOS GENÉRICOS (2/5)

Vantagens tipos genéricos

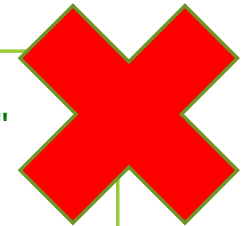
- Evitar criar diferentes implementações para cada tipo

```
var numbers = new ListInt();  
var cities = new ListString();  
var people = new ListPerson();
```



- Evitar criar uma implementação super genérica com object e termos de lidar com “Boxing” e “Downcasting”

```
var objects = new ObjectList();  
objects.Add(new Person()); // Pessoa é guardado como objeto "Boxing"  
  
var person = objects.First();  
var name = person.Name; // ERRO: person é do tipo object.  
  
name = ((Person) person).Name; // Tem de ser feito o "Downcasting"
```



- Boxing: int -> object, Person -> object
- Downcasting: object -> int, object -> Person
- Deve ser validado com “as” ou “is”

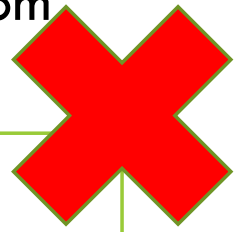
```
Cachorro c = obj as Cachorro;  
if (c != null)  
{  
    c.Latir();  
}
```

TIPOS GENÉRICOS (3/5)

Vantagens tipos genéricos

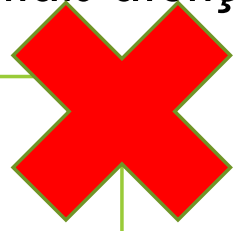
- Diminuir a probabilidade de *bugs* pois não se pode trabalhar com tipos diferentes, a não ser que sejam subtipos.

```
var people = new List<Person>();  
people.Add(new Person());  
people.Add(new Animal()); // ERRO: Não podemos ter animais numa lista de pessoas
```



- Uma lista de objetos é mais permissiva, o que requer mais atenção

```
var objects = new ObjectList();  
objects.Add(new Person()); // podemos guardar pessoas  
objects.Add(new Animal()); // e também animais  
  
var animalOrPerson = objects.First();  
  
if (animalOrPerson is Animal animal)  
    Console.WriteLine($"Animal is in the Forest: {animal.Forest}");
```



TIPOS GENÉRICOS (4/5)

Métodos Genéricos: Exemplo

- O método genérico seguinte permite trocar o conteúdo de duas variáveis

```
public static void Swap<T>(ref T firstObject, ref T secondObject)
{
    T temp = firstObject;
    firstObject = secondObject;
    secondObject = temp;
}
```

podendo ser usado da seguinte forma

```
int x = 10, y = 20;

Swap(ref x, ref y);
// x: 20    y: 10

var p1 = new Person("Jack");
var p2 = new Person("Anna");

Swap(ref p1, ref p2);
// p1.Name: Anna    p2.Name: Jack
```

TIPOS GENÉRICOS (5/5)

Métodos Genéricos: Exemplo

- Como T é um parâmetro do método Swap, não é obrigatório definir o tipo de T entre <>
- O compilador consegue inferir o tipo em causa a partir do tipo dos parâmetros que estão a ser passados.

```
var p1 = new Person("Jack");  
var p2 = new Person("Anna");  
  
Swap(ref p1, ref p2);  
  
// ou  
  
Swap<Person>(ref p1, ref p2);
```

LISTAS GENÉRICAS (1/3)

- Representa uma lista de elementos genérica expansível
- Permite adicionar, inserir e remover elementos
- Tem apenas uma dimensão
- Fortemente tipada, definindo o tipo no momento da inicialização

Declaração e inicialização de listas

```
List<string> colors = new List<string>();  
colors.Add("red");  
colors.Add("green");  
  
var cities = new List<string>() { "Lisboa", "Paris", "Banguécoque" };  
var animals = new List<Animal> {new Hiena(), new Animal()};
```

LISTAS GENÉRICAS (2/3)

Podemos usar o `foreach` para iterar sobre a lista:

- Rápido e fácil de usar iterando sobre todos os elementos
- Elementos ficam disponíveis apenas para operações de leitura (mas as suas propriedades são editáveis)

```
foreach (var city in cities)  
    Console.WriteLine(city);
```

```
foreach (var animal in animals)  
    animal.Nome = animal.Nome.ToUpper();
```

Usando o ciclo `for`

- Mais complexo e flexível, podendo iterar sobre toda a lista ou apenas um subgrupo de elementos
- Elementos ficam disponíveis para leitura e escrita

```
for (var i = 0; i < cities.Count; i++)  
    Console.WriteLine(cities[i]);
```

```
for (var i = 0; i < cities.Count; i++)  
    cities[i] = "";
```

LISTAS GENÉRICAS (3/3)

Métodos mais usados

Add	Adiciona um item no final da lista
AddRange	Adiciona uma coleção de itens no final da lista
Contains	Determina se um elemento existe na lista
Exists	Determina se a lista contém elementos que satisfaçam o predicado e.g. <code>cities.Exists(c => c.StartsWith("c"));</code>
IndexOf	Devolve o índice do primeiro elemento encontrado
Insert	Permite inserir o elemento numa determinada posição
Remove	Remove a primeira ocorrência do objeto com a mesma referência
RemoveAt	Remove o item posicionado num determinado índice
Reverse	Inverte a ordem dos elementos da lista
ToArray	Copia os elementos da lista para um novo array

FILAS GENÉRICAS (1/3)

- Representam uma coleção do tipo FIFO (first-in, first-out)

```
var crepes = new Queue<Crepe>();

crepes.Enqueue(new Crepe("Chocolate"));
crepes.Enqueue(new Crepe("Banana"));
crepes.Enqueue(new Crepe("Morango"));

Console.WriteLine(crepes.Dequeue().ToString()); // Chocolate
Console.WriteLine(crepes.Dequeue().ToString()); // Banana
Console.WriteLine(crepes.Dequeue().ToString()); // Morango

Console.WriteLine(crepes.Dequeue().ToString()); // ERRO: Não há mais crepes!!
```

- Deve validar se há algum elemento na fila antes de o tentar remover

```
if (crepes.TryPeek())
    Console.WriteLine(crepes.Dequeue().ToString());
else
    Console.WriteLine("Chega de gulodices por hoje");
```


FILAS GENÉRICAS (2/3)

- As filas podem ser iteradas com `foreach`, tal como as listas e arrays

```
foreach (var crepe in crepes)
    Console.WriteLine(crepes.ToString());
```

- Mas não é possível aceder aos elementos da fila via índice, invalidando o uso do ciclo `for`

```
for (var i = 0; i < crepes.Count; i++)
    Console.WriteLine(crepes[i].ToString()); // ERRO:
```



FILAS GENÉRICAS (3/3)

Métodos mais usados

Clear	Remove todos os objetos da fila
Contains	Verifica se a fila contém um determinado item
Dequeue	Remove e devolve o elemento que está no início da fila
Enqueue	Adiciona um objeto no final da fila
Peek	Devolve o objeto que está no início da fila (sem o remover)
ToArray	Copia os elementos da fila para um array
TryPeek	Se a fila estiver vazia, devolve false. Caso contrário, devolve true e permite o acesso ao elemento através de referência via parâmetro
TryDequeue	Se a fila estiver vazia, devolve false. caso contrário, remove o elemento da fila, devolve true e permite o acesso ao elemento através de referência via parâmetro

PILHAS GENÉRICAS (1/3)

- Representam uma coleção do tipo LIFO (last-in, first-out)

```
var crepes = new Stack<Crepe>();

crepes.Push(new Crepe("Chocolate"));
crepes.Push(new Crepe("Banana"));
crepes.Push(new Crepe("Morango"));

Console.WriteLine(crepes.Pop().ToString()); // Morango
Console.WriteLine(crepes.Pop().ToString()); // Banana
Console.WriteLine(crepes.Pop().ToString()); // Chocolate

Console.WriteLine(crepes.Pop().ToString()); // ERRO: Não há mais crepes!!
```

- Deve validar-se se há algum elemento na pilha antes de o tentar remover

```
// O que acontece se houver crepes? e se não houver?
if (crepes.TryPop(out var crepe))
    Console.WriteLine(crepe.ToString());
```

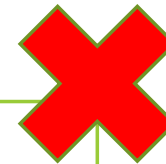
PILHAS GENÉRICAS (2/3)

- As pilhas podem ser iteradas com foreach, tal como as listas e arrays

```
foreach (var crepe in crepes)
    Console.WriteLine(crepes.ToString());
```

- Mas não é possível aceder aos elementos da pilha via índice, invalidando o uso do ciclo for

```
for (var i = 0; i < crepes.Count; i++)
    Console.WriteLine(crepes[i].ToString()); // ERRO
```



PILHAS GENÉRICAS (3/3)

Métodos mais usados

Pop	Remove e devolve o item no topo da pilha
Push	Insere um item no topo da pilha
TryPeek	Se a pilha estiver vazia, devolve false. Caso contrário, devolve true e permite o acesso ao elemento através de referência via parâmetro
TryPop	Se a pilha estiver vazia, devolve false. caso contrário, remove o elemento da fila, devolve true e permite o acesso ao elemento através de referência via parâmetro
Remove	Remove todos os elementos da pilha
Contains	Determina se um determinado objeto existe na pilha
Count	Devolve o número total de elementos na pilha
ToArray	Copia os elementos da pilha para um novo array

DICIONÁRIOS (1/7)

- Coleção fortemente tipada com par chave-valor
- Especialmente útil para acesso rápido a um elemento através da chave
- Chave:
 - Deve ser única
 - Não pode ser alterada
 - Não pode ser null

Key	Value
"LX"	"Lisboa"
"PAR"	"Paris"
"MA"	"Madrid"

DICIONÁRIOS (2/7)

Listas versus Dicionários

Lista	Dicionário
Contém elementos	Contém elementos definidos como pares chave-valor
Acesso por índice	Acedidos por chave
Permite elementos duplicados	Permite elementos duplicados mas chaves únicas
O desempenho tende a piorar ao pesquisar um elemento, à medida que a lista vai crescendo	Desempenho relativamente linear na pesquisa utilizando a chave

DICIONÁRIOS (3/7)

Podemos usar qualquer combinação de tipos para o par chave-valor usando a classe `Dictionary<TKey, TValue>`

- `Dictionary<int, int>`
- `Dictionary<string, Crepe>`
- `Dictionary<int, Veiculo>`

```
Dictionary<string, decimal> exchangeRates = new Dictionary<string, decimal>();  
exchangeRates.Add("EURUSD", 1.16m);  
exchangeRates.Add("USDEUR", 0.86m);  
exchangeRates.Add("BRLEUR", 4.37m);  
exchangeRates.Add("EURBRL", 0.22m);
```

```
// Usando var e inicializadores de coleções  
var exchangeRates = new Dictionary<string, decimal>  
{  
    {"EURUSD", 1.16m},  
    {"USDEUR", 0.86m},  
    {"BRLEUR", 4.37m},  
    {"EURBRL", 0.22m}  
};
```


DICIONÁRIOS (4/7)

```
foreach (var exchangeRate in exchangeRates)
{
    // Exemplo de utilização do Deconstruct (Não confundir com Destrutor)
    var (key, value) = exchangeRate;
    Console.WriteLine($"Chave:{key}, Valor:{value}");
    // Utilizando o objeto
    Console.WriteLine($"Chave:{exchangeRate.key}, Valor:{exchangeRate.value}");
}
```

```
// Iteração sobre a chave do dicionário
foreach (var key in exchangeRates.Keys)
{
    // Pesquisa por chave
    var value = exchangeRates[key];
    Console.WriteLine($"Chave:{key}, Valor:{value}");
}
```

```
// Iteração sobre os valores do dicionário
foreach (var value in exchangeRates.Values)
{
    // Qual o problema de desempenho com esta abordagem?
    var element = exchangeRates.First(er => er.Value == value);
    Console.WriteLine($"Chave:{element.Key}, Valor:{value}");
}
```

Atenção: caso exista mais que um valor igual vai devolver mais que um elemento, daí a utilização do First().

DICIONÁRIOS (5/7)

Os dicionários não permitem o uso de índices de forma explícita

```
for (var i = 0; i < exchangeRates.Count; i++)  
{  
    // ERRO: É esperado o valor da chave do tipo string  
    var currency = exchangeRates[i];  
  
    Console.WriteLine($"Currency:{currency}");  
}
```



```
for (var i = 0; i < exchangeRates.Count; i++)  
{  
    // OK. Usando métodos de extensão LINQ  
    var currency = exchangeRates.ElementAt(i);  
  
    Console.WriteLine($"Currency:{currency}");  
}
```

```
var dictionary = new Dictionary<int, string> {{0, "a"}, {1, "b"}};  
  
// E agora, funciona? Porquê?  
for (var i = 0; i < dictionary.Count; i++)  
{  
    Console.WriteLine($"Currency:{dictionary[i]}");  
}
```

DICIONÁRIOS (6/7)

Métodos mais usados

Add	Adiciona um elemento par chave-valor
Clear	Remove todas as chaves e valores do dicionário
ContainsKey	Determina se o dicionário tem uma chave específica
ContainsValue	Determina se o dicionário tem um valor específico
Remove	Remove o elemento do dicionário
TryAdd	Tenta adicionar um elemento ao dicionário. Especialmente útil se houver probabilidade de inserir uma chave duplicada ou com valor null (apenas na .Net Core)
TryGetValue	Devolve o valor associado à chave passada por parâmetro (quando encontrado)

Boa prática: Use os métodos começados com Try quando não tiver a certeza que pode fazer essa operação sem que seja lançada uma exceção (a não ser que esse seja o comportamento desejado)

DICIONÁRIOS (7/7)

```
var exchangeRates = new Dictionary<string, decimal>();  
decimal value;  
  
if (!exchangeRates.TryGetValue("EUR", out value))  
    value = 0;
```

```
var exchangeRates = new Dictionary<string, decimal>();  
decimal value;  
  
if (exchangeRates.ContainsKey("EUR"))  
    value = exchangeRates["EUR"];  
else  
    value = 0;
```

```
var exchangeRates = new Dictionary<string, decimal>();  
  
if (!exchangeRates.ContainsKey("EUR"))  
    exchangeRates.Add("EUR", (decimal)1.5);
```

Tenta obter o valor de uma determinada Key verificando antes se a mesma existe.

Adiciona um novo elemento apenas se ainda não existir, evitando assim erros.

LISTAS E DICIONÁRIOS ORDENADOS

Podemos criar listas ou dicionários que se ordenam automaticamente pela chave. Funciona de forma muito idêntica ao Dictionary.

- SortedList<int, string>
 - Ideal para listas com poucos dados
 - Permite acesso por índice
- SortedDictionary<int, Crepe>
 - Ideal para uma grande quantidade de dados

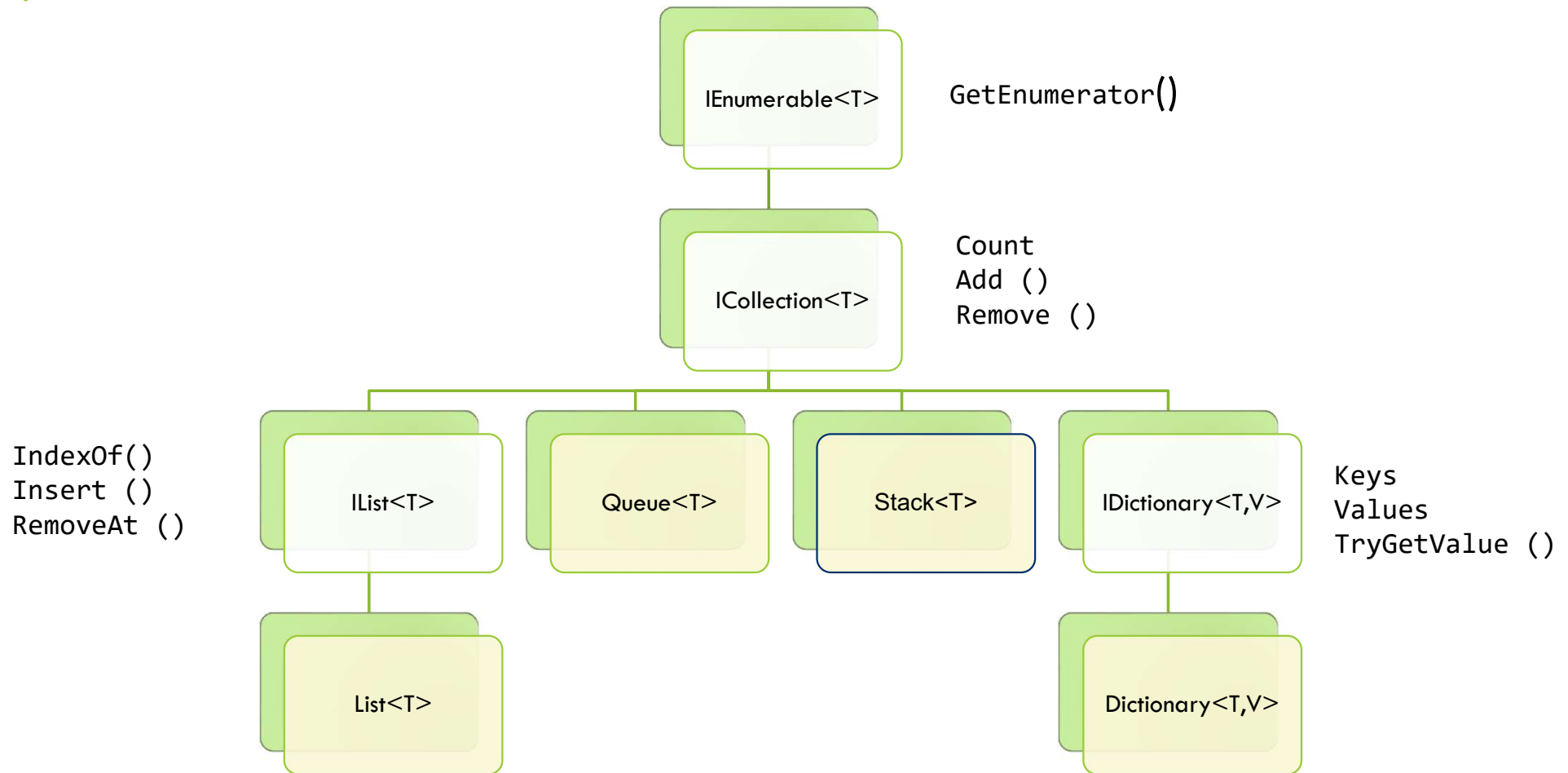
```
var lista = new SortedList<int, string>();

lista.Add(3, "C");
lista.Add(1, "A");
lista.Add(2, "B");

foreach (var item in lista)
{
    Console.WriteLine($"{item.Key} - {item.Value}");
}

// Output: 1 - A, 2 - B, 3 - C
```

HIERARQUIA DE INTERFACES



INTERFACES

- Consiste num contrato em que se define o que uma classe deve fazer mas não como fazer.
- Não contém implementação.
- Contém apenas a definição dos métodos e propriedades a implementar na classe.
- Permite herança múltipla, ou seja, uma classe derivar de várias interfaces
- Nomenclatura começa sempre por um “I” maiúsculo.

```
public interface IAnimal
{
    0 referências
    string Nome { get; set; }
    0 referências
    void FazerSom();
}
```

IENUMERATOR E IENUMERABLE (1/5)

- A interface IEnumerable representa um objeto que pode ser enumerado e.g. a classe do tipo List, definindo apenas o método GetEnumerator

```
public interface IEnumerable
{
    IEnumerator GetEnumerator();
}
```

- O método GetEnumerator devolve um objeto do tipo IEnumerator, que pode ser usado para iterar sobre um determinado objeto

```
public interface IEnumerator
{
    bool MoveNext();
    object Current { get; }
    void Reset();
}
```


IENUMERATOR E IENUMERABLE (2/5)

- Com a implementação destas interfaces, o cliente pode usar o método `MoveNext()` para iterar um determinado objeto e usar a propriedade `Current` para aceder a um elemento de cada vez
- Como a lista de crepes implementa a interface `IEnumerable`, podemos percorrer cada um dos elementos desta forma

```
var enumerator = crepes.GetEnumerator();  
while (enumerator.MoveNext())  
{  
    Console.WriteLine(enumerator.Current.ToString());  
}
```

- O `foreach` executa esse mesmo código, providenciando uma forma limpa de percorrer uma sequência de elementos

```
foreach (var crepe in crepes)  
{  
    Console.WriteLine(crepe.ToString());  
}
```

IENUMERATOR E IENUMERABLE (3/5)

- A implementação da interface IEnumerable faz com que possamos usar o foreach de forma consistente em todas as coleções (Filas, Listas, Pilhas, Dicionários,...)
- Em determinados cenários, permite otimizar o código em várias ordens de grandeza. Como implementaria código para:
 - Processar um ficheiro com milhares de ordens de encomenda
 - Criar um método para devolver 10 milhões de números da sequência de Fibonacci (1, 1, 2, 3, 5, 8, 13...)

Será que a memória do seu PC gostaria de ter todos esses elementos em memória?

```
public IEnumerable<string> LerLinhas(string caminho)
{
    using (var reader = new StreamReader(caminho))
    {
        string linha;

        while ((linha = reader.ReadLine()) != null)
        {
            yield return linha;
        }
    }
}
```

```
foreach (var linha in LerLinhas("ficheiro.txt"))
{
    Console.WriteLine(linha);
}
```

IENUMERATOR E IENUMERABLE (4/5)

- Quais as diferenças entre os exemplos abaixo?

```
public IList<int> NumerosPares(int quantidade)
{
    var numerosPares = new List<int>(quantidade);
    var numeroAtual = 0;

    while (numerosPares.Count <= quantidade)
    {
        if (++numeroAtual % 2 == 0)
            numerosPares.Add(numeroAtual);
    }

    return numerosPares;
}
```

Valores têm de ser todos carregados para memória.

```
public IEnumerable<int> NumerosParesOtimizado(int quantidade)
{
    var numeroAtual = 0;

    while (++numeroAtual <= quantidade)
    {
        if (numeroAtual % 2 == 0)
            yield return numeroAtual;
    }
}
```

Valores são obtidos em cada iteração, evitando a implementação do IEnumerator. A cada “Yield return” o valor é retornado e a execução para nesse local.

IENUMERATOR E IENUMERABLE (5/5)

- Considere devolver um `IEnumerable<T>` providenciando classes imutáveis
 - Classes que permitem percorrer os elementos mas não adicionar ou alterar elementos
- Considere devolver um `IEnumerable<T>` quando os casos de uso são desconhecidos
 - Caso não se saiba como o utilizador vai usar os dados, permitindo a utilização de LINQ para por exemplo ordenar, filtrar, transformar, etc.
- Evite devolver um `IEnumerable<T>` se:
 - O código cliente pretende modificar a coleção
 - É necessário aceder a propriedades de coleção, e.g. `Count`

ICOLLECTION

- A interface ICollection deriva de IEnumerable e permite criar uma outra camada de abstracção, definindo os métodos:
 - Count
 - Add
 - Remove
- Considere usar ICollection em vez de IEnumerable sempre que precisar de usar um dos métodos acima

HIERARQUIA DE INTERFACES

